Phala Network TEE 구현체 분석 - Pink Runtime (스마트 컨트랙트 실행 환경)

1. Pink Runtime 개요

Pink Runtime은 Phala Network의 **TEE 내 스마트 컨트랙트 실행 환경**으로, Substrate의 pallet-contracts를 기반으로 하여 TEE 특화 기능을 추가한 런타임입니다.

2. 핵심 구성 요소

2.1 Substrate 팰릿 통합

```
# 핵심 Substrate 팰릿

├── pallet-contracts: 스마트 컨트랙트 실행 엔진

├── pallet-balances: 잔액 관리

├── pallet-timestamp: 시간 관리

├── pallet-insecure-randomness-collective-flip: 랜덤성 제공

├── frame-support: 프레임워크 지원

└── frame-system: 시스템 기본 기능
```

2.2 Pink 특화 컴포넌트

```
# Pink 생태계
```

```
├── pink: Pink 런타임 코어
├── pink-chain-extension: 체인 확장 기능
├── pink-capi: C API 인터페이스
├── phala-crypto: 암호화 기능
└── phala-types: 타입 정의
```

3. 라이브러리 타입 및 빌드

3.1 크레이트 타입

```
[lib]
crate-type = ["lib", "cdylib"]
```

- **lib**: 정적 라이브러리로 링크
- **cdylib**: 동적 라이브러리로 로드 (런타임 로딩용)

3.2 특수 기능

- **allocator**: 커스텀 메모리 할당자 사용
- **sanitized-logger**: 보안 로깅 시스템

4. 스마트 컨트랙트 실행 환경

4.1 WASM 실행 엔진

- **Substrate Contracts**: pallet-contracts 기반
- **WASM 런타임**: WebAssembly 바이트코드 실행

- **Gas 미터링**: 실행 비용 측정 및 제한
- **메모리 관리**: 안전한 메모리 할당 및 해제

4.2 상태 관리

```
// 상태 관리 컴포넌트 (추정 구조)
├── State Machine: sp-state-machine 통합
├── Externalities: sp-externalities 환경
├── Trie Storage: sp-trie 기반 저장소
└── Hash Database: hash-db 해시 저장소
```

5. Chain Extension 시스템

5.1 TEE 특화 확장

- **HTTP 요청**: 외부 API 호출 기능
- **암호화 연산**: TEE 내 암호화 서비스
- **랜덤성**: 안전한 난수 생성
- **시간 서비스**: 신뢰할 수 있는 타임스탬프

5.2 확장 기능 구현

```
// Chain Extension 예시 (추정)
impl ChainExtension for PinkExtension {
fn  call(&mut  self, env: Environment) -> Result<RetVal, DispatchError> {
```

```
match  env.func_id() {
HTTP_REQUEST => self.http_request(env),
CRYPTO_SIGN => self.crypto_sign(env),
GET_RANDOM => self.get_random(env),
// ... 기타 확장 기능
}
}
}
```

6. 개발 및 테스트 환경

6.1 테스트 도구

```
[dev-dependencies]
├── insta: 스냅샷 테스트
├── hex-literal: 16진수 리터럴
├── env_logger: 환경 로깅
├── assert_matches: 패턴 매칭 어서션
├── serde: 직렬화 테스트
├── im: 불변 데이터 구조
├── rusty-fork: 포크 테스트
└── ink: Ink! 스마트 컨트랙트 프레임워크
```

6.2 테스트 컨트랙트

- **check_system**: 시스템 기능 검증 컨트랙트
- **ink-as-dependency**: Ink! 의존성 테스트

7. 보안 및 격리

7.1 메모리 보안

- **커스텀 할당자**: phala-allocator 사용
- **메모리 격리**: 컨트랙트 간 메모리 분리
- **오버플로우 방지**: 안전한 산술 연산

7.2 실행 격리

- **WASM 샌드박스**: WebAssembly 격리 환경
- **리소스 제한**: CPU, 메모리 사용량 제한
- **권한 제어**: 시스템 리소스 접근 제어

8. 런타임 로딩 시스템

8.1 동적 로딩

- **cdylib**: 동적 라이브러리 형태로 빌드
- **런타임 교체**: 무중단 런타임 업그레이드
- **버전 관리**: 런타임 버전 호환성 관리

8.2 로딩 프로세스

```
1. 런타임 바이너리 로드

2. 심볼 해석 및 링크

3. 초기화 함수 실행

4. API 인터페이스 등록

5. 실행 준비 완료
```

9. 성능 최적화

9.1 컴파일 최적화

- **LTO**: 링크 타임 최적화
- **코드 생성**: 최적화된 WASM 코드 생성
- **인라인**: 함수 인라인 최적화

9.2 런타임 최적화

- **캐싱**: 컴파일된 WASM 코드 캐싱
- **프리컴파일**: 자주 사용되는 코드 사전 컴파일
- **메모리 풀**: 메모리 할당 최적화

10. 디버깅 및 모니터링

10.1 로깅 시스템

- **tracing**: 구조화된 로깅
- **sanitized-logger**: 보안 로그 필터링

- **레벨 제어**: 로그 레벨 동적 조정

10.2 개발 도구

- **git-revision**: 빌드 버전 추적
- **wasm-checker**: WASM 코드 검증
- **this-crate**: 크레이트 메타데이터

이 문서는 Pink Runtime의 스마트 컨트랙트 실행 환경을 다룹니다. 다음 문서에서는 SGX Attestation 시스템을 분석합니다.